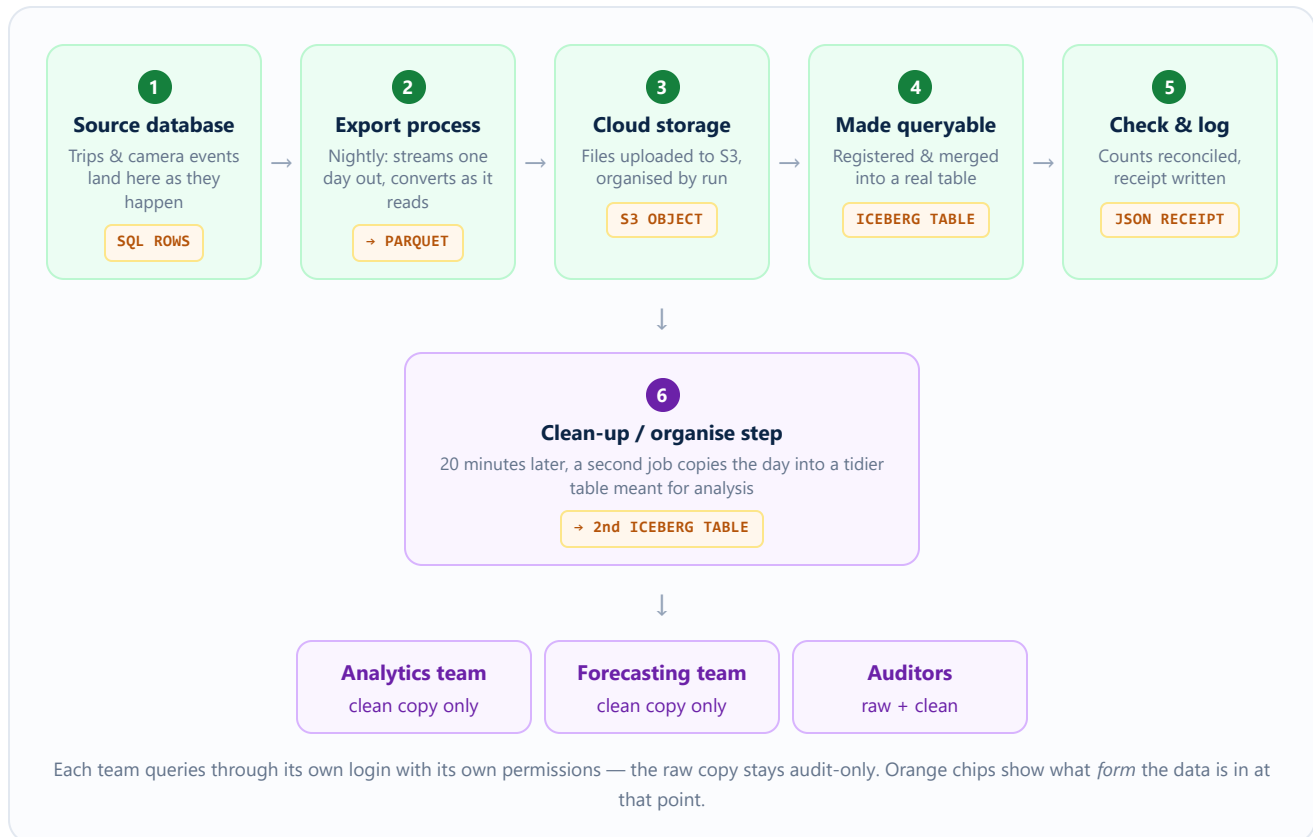


How Data Moves Through the System

What actually happens to a single trip record — from the moment it's written to the operational database, to the moment an analyst queries it — and what form the data takes at each hop. As of 21 September 2026.



● Built & working, runs on its own schedule ● Partly done ● Not built yet

The short version: rows leave Postgres one day at a time, get rewritten into a compressed columnar file, land in cloud storage, get registered into a table format that supports versioning and row-level deletion, get checked against the source, and then get copied once more into a cleaner table that analysts are allowed to touch. The pages that follow walk each hop in detail.

Step by Step — What Happens and Why

1

Source database Working

Trip and camera events get written to a Postgres database as they happen — this is the day-to-day operational system, not part of the lake itself.

Right now: a realistic practice database (`mtsai-api-sim`) stands in for the real one, since a connection to the actual production database doesn't exist yet. Same shape, same behaviour, synthetic data — about a year of history, roughly 35,000 trips.

2

Export process Automatic

Once a night, a small program wakes up on its own, connects to the database, and asks for everything that happened "yesterday" — one full day of rows.

How the data actually moves: it doesn't pull the whole day into memory at once — it streams rows out in batches of 1,000 using a database cursor, the same technique used for reports too large to fetch in one go. Each batch is converted into a compact, column-oriented format (Parquet) as it arrives, so memory use stays flat no matter how big the day is.

3

Cloud storage Automatic

That Parquet file gets uploaded to cloud storage (S3) — cheap, durable, and organised so a specific day's data can be found without scanning everything.

Where it lands: first into a staging area under a folder named after that run, then — once the next step succeeds — into a `raw/` area that holds an unmodified, audit-grade copy of exactly what the source database contained that day. Nothing is cleaned or corrected here on purpose, so it can always be trusted as the original record.

4

Made queryable Automatic

Uploaded files are just files until something tells the query engine they exist. This step turns them into a table people can run plain SQL against.

Under the hood: the staged file is briefly registered as a temporary table, its rows are inserted into the permanent one, and the temporary registration is dropped again. If that same day is ever re-processed, the previous copy is deleted first — so re-runs replace, they never duplicate. That guarantee is the reason a failed night can simply be run again without anyone checking for damage.

5

Check & log Automatic

Before calling the night's work done, the system checks its own results — did the number of rows that landed in the lake match what was actually read from the source, and do their ID totals agree?

The paper trail: a small receipt file is written recording exactly what happened — row count, checksum, how long it took, success or failure. If the two sides disagree, the run is marked failed and an alert fires rather than silently moving on with quietly wrong data.

6

Clean-up / organise step Automatic

About 20 minutes after the export finishes, a second, separate process copies that day's data from the raw area into a second, tidier table meant for actual analysis.

Why a separate step: keeping "the exact original" and "the easy-to-use version" as two different tables means analysts get something clean to work with, while auditors always still have an untouched original to check against. Today the two hold the same columns — the clean copy is a faithful duplicate, not yet reshaped — so the real difference right now is who's allowed to read which.

Everything above runs unattended. Each "Automatic" step has been watched firing on its own schedule at least once — not merely configured and assumed to work.

Where to Look, What You'll See, and How the Data Changes Shape

Everything below is in the AWS Console, region **ap-south-1**, account **690293068614**. "Shape" means what form the data is physically in at that point — the same trip record is stored very differently at each hop.

1 · Source database — where it starts

- WHERE** RDS → Databases → `mtsai-api-sim-test` . To actually read rows, use `tools/pg-query` in the repo.
- WHAT YOU'LL SEE** An ordinary relational table, `trip_events` — one row per trip, with columns for trip ID, a hashed vehicle ID, city code, event date, distance and fare.
- SHAPE** **Row-oriented SQL rows.** Each trip is stored as one contiguous record, which is what makes "fetch this single trip" fast and "average every fare this year" slow.

2 · Export reads and rewrites it

- WHERE** CloudWatch → Log groups → `/mtsai-datalake/test/export` → a stream named `export/export/<task-id>` .
- WHAT YOU'LL SEE** A short, readable trace of the night: `read 76 row(s) from postgres for 2026-07-07` , then `uploaded 76 row(s) to s3://.../data.parquet` , then `succeeded: 76 row(s), checksum=177962, duration=13.79s` .
- SHAPE CHANGE** **SQL rows → Parquet columns.** Those 76 trips stop being 76 separate records and become 7 columns of 76 values each, compressed column by column. One quirk worth knowing: the date is deliberately written as a plain integer (days since 1970) because the Parquet library mis-encodes real date values — a bug caught only when a date came back as `-1454296-02-29` .

3 · Landing in cloud storage

- WHERE** S3 → `mtsai-datalake-test-690293068614-ap-south-1` → `raw/_staging/trip_events/<run-id>/`
- WHAT YOU'LL SEE** One folder per run, named with that run's unique ID, holding a single `data.parquet` file — typically only a few kilobytes for a normal day, since columnar compression is very effective on this kind of data.
- SHAPE** **A single binary object.** At this moment it is emphatically *not* a table — it's an opaque file sitting in storage that no query engine knows anything about yet.

4 · Becoming a real, queryable table

- WHERE** S3 → `raw/trip_events/` · Athena → Query editor → database `test_raw` · Glue → Tables
- WHAT YOU'LL SEE** In S3, two sibling folders appear: `data/` holds the Parquet files the table is actually made of, and `metadata/` holds JSON and Avro files tracking which data files belong to which version of the table. In Athena, `SELECT * FROM trip_events LIMIT 10` now returns rows.
- SHAPE CHANGE** **Loose file → Iceberg table (data + metadata).** This is the most significant transformation in the whole chain. The file stops being just bytes and gains a metadata layer that records "as of this snapshot, the table consists of exactly these files." That layer is what later makes two things possible that plain files can't do: looking at the table as it was last week, and deleting one person's rows out of the middle of it.

5 · The receipt that proves it worked

- WHERE** S3 → `export-manifests/2026-07-07/trip_events.json` (one file per table, per day)
- WHAT YOU'LL SEE** A small JSON record: which table, which date, the run's unique ID, how many rows landed, the checksum, how long it took, and whether it succeeded. Backfills also write a range-level summary under `export-manifests/backfill/`.
- SHAPE** **A summary record — not the data itself.** Nothing about the trips changes here. This is the evidence trail an auditor can read without touching the data, and it's the reason a failed night is visible rather than silent.

6 · The clean copy analysts actually use

- WHERE** S3 → `curated/trip_events/` · Athena → database `test_curated` · logs in the same log group under `curate/`
- WHAT YOU'LL SEE** A second Iceberg table with its own `data/` and `metadata/` folders, holding the same day's rows a second time — physically separate files, not a view or a pointer.
- SHAPE CHANGE** **Iceberg (raw) → Iceberg (curated).** A cross-database insert reads from raw and writes brand-new files into curated. The columns are identical today, so this is currently a duplication rather than a reshaping — the meaningful change is *governance*, not structure: raw stays audit-only, curated is what analytics and forecasting are permitted to read.

7 · Someone finally queries it

- WHERE** Athena → Query editor (pick the workgroup for your team) · results land in S3 under `athena-results/<workgroup>/`
- WHAT YOU'LL SEE** Results in the console, and a `.csv` file per query in S3 that expires automatically after 7 days so query output doesn't accumulate forever.
- SHAPE CHANGE** **Iceberg table** → **CSV result set**. Athena reads the metadata layer first to work out which Parquet files it even needs to open, scans only those, and writes the answer out as ordinary CSV. Skipping irrelevant files is what keeps query cost proportional to the question rather than to the size of the lake.

Weekly tidy-up — keeping the metadata from growing forever

- WHERE** Same log group, streams starting with `compact/` · runs every Sunday 03:00 UTC
- WHAT YOU'LL SEE** Log lines confirming each table was vacuumed, plus four storage-size numbers published as metrics (one per area of the bucket) that feed the cost dashboard.
- SHAPE CHANGE** **Fewer files, same rows**. Old table versions are expired and the files only they referenced are deleted. Nothing visible changes for anyone querying — the current version of the table is untouched.

Erasing one person's data on request

- WHERE** Same log group, streams starting with `erasure/` · receipts in S3 under `export-manifests/erasure/`
- WHAT YOU'LL SEE** A record per request: the reference, the identifier, how many rows were found and removed from each table, and confirmation that the old version can no longer be reached.
- SHAPE CHANGE** **Rows removed from the present *and* the past**. The rows are deleted from both tables, which writes a new table version without them — but the previous version would still contain them, so the old versions are then destroyed outright. Verified by trying to read the pre-deletion version afterwards and confirming it no longer exists at all.

Costs and alerts: CloudWatch → Dashboards → `mtsai-datalake-test-cost` shows storage by area, query bytes scanned per team, and how long the nightly jobs run. Failures and budget overruns email `siby@miracletraffic.ai`.

What's Automatic Today vs. What's Not

Capability	Status	What that means
1. Data exists in the source database	Working	Real for testing purposes; the actual production database isn't connected yet
2. Export process runs	Automatic	Runs on its own every night — confirmed by watching it fire without anyone starting it
3-5. Storage, cataloguing, checking, logging	Automatic	Fully automatic, no manual work needed once step 2 starts
6. Clean-up / organise step	Automatic	Runs on its own every night, 20 minutes after step 2
7. Teams querying the data	Working	Access rules built and tested by actually trying to read as each team
Catching up on older data	Partly done	A "process many days at once" mode exists and has been run for real over a multi-week range; hasn't been run over the full available history yet
Deleting one person's data on request	Working	Rehearsed twice against real data — removed from every table and made unrecoverable, not just hidden
Weekly storage tidy-up	Automatic	Runs every Sunday, expires old table versions and publishes storage-size numbers
Cost visibility & alerts	Working	A live dashboard plus email alerts if a nightly run fails or spending crosses a threshold

The one honest caveat: everything above is real, running infrastructure — but the source database is still a realistic stand-in rather than the production one. Every shape change, every table, and every check described in this document is genuine; what's synthetic is the trips themselves.

Generated from the current state of the mtsai-datalake project, 21 September 2026. Reflects what's actually built and tested, not the eventual full design. Companion to [MTSAi-Data-Lake-Build-Report.pdf](#) and [MTSAi-Data-Lake-Phase3-Build-Report.pdf](#) for the technical detail behind each claim.